

Relatório comparativo e custo de transformação

Stacks InfoLAB (referência) vs Infotime Web — SaaS multi-tenant em escala

Versão: 1.1

Data de referência: maio de 2026

Âmbito: comparação técnica, eficiência operacional, custo relativo de migração entre padrões.

1. Objetivo do documento

Consolidar diferenças de stack (front-end, back-end, BFF, validação, auth), discutir **eficiência** num cenário SaaS com **muitos tenants e muitos utilizadores por tenant**, e estimar o **custo de transformação** para:

- alinhar o **Infotime** ao padrão **InfoLAB**;
- alinhar o **InfoLAB** ao padrão **Infotime**.

Nota sobre “custo”: neste relatório, custo é expresso sobretudo em **esforço (pessoa-semana / pessoa-mês)** e **risco**, não em valores monetários absolutos. A conversão para euros ou dólares depende da tarifa interna e do calendário.

2. Recapitulação das diferenças de stack

2.1 Monorepo e tooling

Aspeto	Infotime Web	InfoLAB
Pacotes	pnpm workspaces	npm workspaces + Nx
Orquestração	Scripts pnpm	nx run-many, cache Nx
Node (referência)	—	≥ 22 declarado

2.2 Front-end

Aspeto	Infotime	InfoLAB
Runtime	Vite 6 + React 18 + React Router 7 (SPA)	Next.js 16 + React 19 (App Router)
Dados	TanStack Query	Sem React Query explícito no manifest
Formulários	react-hook-form + Zod	Componentes “Liga”; validação variada
i18n	Não no manifest	next-intl
UI	PrimeReact + PrimeFlex + CSS Liga	PrimeReact + CSS Liga
Testes web	Lint placeholder	Vitest + Testing Library

2.3 Back-end

Aspeto	Infotime API	InfoLAB API
--------	--------------	-------------

NestJS	Sim	Sim
HTTP	Fastify	Express
Validação	Zod (pipe global)	class-validator / class-transformer
Prisma	Pacote <code>@infotime/database</code>	Prisma dentro do projeto <code>api</code>
Auth típica	JWT / SPA	Cookies HTTP-only , proxy/BFF
Extras	BullMQ, nestjs-cls	Throttler + Redis, etc.
Testes API	Vitest	Jest + Supertest

2.4 BFF

- **InfoLAB:** Route Handlers (`app/api`), login, sessão, proxy para Nest.
- **Infotime:** sem Next; cliente → API direta (proxy só em dev no Vite).

3. Eficiência em SaaS multi-tenant (síntese)

Grandes volumes de tenants e utilizadores pressionam sobretudo:

- modelo de **isolamento na BD** (tenant_id, RLS, quotas);
- **pool de conexões** e Postgres;
- **cache** e **rate limiting** por tenant;
- **observabilidade** por tenant.

Framework HTTP (Express vs Fastify) e **SPA vs Next** são secundários face ao **desenho de dados e auth**, desde que a API escale horizontalmente e o pool DB esteja correto.

- **SPA + API direta + JWT** tende a **menos hops** e deploy simples (CDN + API).
- **Next + BFF + cookies** tende a **melhor superfície** contra XSS no token em memória do JS, ao custo de **mais um tier** e complexidade operacional.

4. Custo para transformar Infotime no padrão InfoLAB

4.1 O que “padrão InfoLAB” implica (escopo típico)

Inclui, para um alinhamento forte:

Faixa	Conteúdo
Núcleo	Migrar apps/web de Vite SPA para Next.js App Router ; estrutura <code>app/</code> ; layouts; substituir React Router por rotas Next.
BFF	Implementar Route Handlers equivalentes (auth, proxy <code>/api</code> , sessão, permissões) alinhados ao Nest InfoLAB.
Auth	Passar de fluxo JWT no cliente para cookies HTTP-only + endpoints de sessão (impacto em CORS, SameSite, domínios).
i18n	Introduzir next-intl (mensagens, rotas por locale se necessário).

UI Liga	Portar ou recriar padrões (LigaFormularioBase, listagens, abas, etapas) no modelo Next (Client/Server Components).
API	Manter Nest; opcionalmente alinhar Express em vez de Fastify se quiser paridade total com o repositório referência (não obrigatório para “parecer InfoLAB” no front).
Validação	Opcional: duplicar contratos em class-validator ou manter Zod e gerar DTOs — decisão de custo.
Testes	Vitest/RTL no web; e2e se política de QA exigir.

4.2 Estimativa de esforço (ordem de grandeza)

Premissas: equipa com experiência em Next e Nest; escopo **aplicação Infotime atual** (módulos já existentes), não “clonar infolab-web inteiro”.

Fase	Descrição resumida	Ordem de grandeza
F0 — Descoberta	Mapear rotas, auth, proxy, envs, parity com InfoLAB	2–4 pessoa-semanas
F1 — Shell Next	Next 16, layout shell, tema Liga, login, home com abas	4–10 pessoa-semanas
F2 — BFF + auth cookie	Route Handlers, proxy Nest, sessão, migração de fluxo de login	6–14 pessoa-semanas
F3 — Módulos	CRUD/listagens/formulários no App Router + padrões Liga	8–20+ pessoa-semanas (varia com número de telas)
F4 — i18n	next-intl, extração de strings	3–8 pessoa-semanas
F5 — Qualidade	Testes, hardening, observabilidade, doc	4–12 pessoa-semanas

Total indicativo (projeto médio já funcional): ~27–68 pessoa-semanas (~7–17 pessoa-meses FTE), antes de contingência.

Contingência recomendada: +25–40% por integração auth/BFF, regressões e alinhamento com segurança.

4.3 Principais fatores que aumentam o custo

- Paridade **visual e comportamental** com **todo** o corpus de componentes InfoLAB (dezenas de módulos).
- **Duplicar** Nest para Express só por homogeneidade (ganho marginal vs custo).
- Ambientes **multi-domínio** (cookie entre subdomínios).
- Equipa sem experiência prévia em **Next App Router** ou **BFF**.

4.4 Risco

Risco	Impacto
Regressões em auth e sessão	Alto — utilizadores bloqueados
Desempenho se BFF fizer lógica pesada	Médio
Duplicação de validação API/front	Médio (manutenção)

5. Custo para transformar InfoLAB no padrão Infotime

5.1 O que “padrão Infotime” implica (escopo típico)

Faixa	Conteúdo
Núcleo	Substituir Next por Vite + React Router ; remover dependência de <code>app/</code> e Route Handlers para UI principal.
API	Consumidor direto do Nest (Fastify opcional): eliminar ou reduzir BFF Next; JWT (ou Bearer) no cliente com política CSP rigorosa.
Auth	Migrar de cookie HttpOnly via Next para token + armazenamento seguro (preferência: cookie HttpOnly emitido pela própria API num mesmo site, ou refresh flow) — design explícito.
Validação	Opcional: Zod partilhado e pipe Nest alinhado ao Infotime.
Estado servidor	Perder SSR nas áreas que o usam; marketing pode ficar em site estático separado.
i18n	Substituir next-intl por i18next ou similar, ou fase única de idioma.
UI	Manter CSS/componentes Liga mas sem <code>dynamic()</code> do Next — usar <code>React.lazy</code> / imports estáticos.

5.2 Estimativa de esforço

Fase	Descrição resumida	Ordem de grandeza
F0 — Descoberta	Inventário <code>app/api</code> , dependências de cookie, rotas	2–4 pessoa-semanas
F1 — SPA shell	Vite, shell, router, tema, login JWT	4–8 pessoa-semanas
F2 — Remover BFF	Apontar front à API; CORS; substituir fluxos auth	5–12 pessoa-semanas
F3 — Módulos	Migrar páginas Next para rotas SPA (grandes blocos com <code>dynamic import</code> no InfoLAB)	10–25+ pessoa-semanas
F4 — i18n / docs	Migração de <code>next-intl</code> ; markdown se aplicável	2–8 pessoa-semanas
F5 — Qualidade	Testes (TanStack Query), segurança XSS, doc	4–10 pessoa-semanas

Total indicativo: ~27–59 pessoa-semanas (~7–15 pessoa-meses FTE), + contingência 20–35%.

5.3 Principais fatores que aumentam o custo

- Volume enorme de **dynamic imports** e rotas em `app/` no InfoLAB.
- Funcionalidades que **dependem** de Route Handlers (permissões, proxy genérico).
- Requisito de **manter cookies** sem Next (exige API ou gateway a emitir cookie).

5.4 Risco

Risco	Impacto
Segurança XSS se JWT em storage errado	Alto
Perda de funcionalidades SSR/SEO onde eram críticas	Médio

6. Comparação lado a lado dos custos de transformação

Dimensão	Infotime → InfoLAB	InfoLAB → Infotime
Mudança de paradigma	SPA → Next + BFF (maior curva)	Next → SPA (grande refator de rotas)
Auth	Cookies + BFF (reengenharia)	JWT/API direta (reengenharia)
Estimativa central	~10–14 pessoa-meses + contingência	~8–12 pessoa-meses + contingência
Pior caso (paridade ampla UI)	18–24+ pessoa-meses	15–20+ pessoa-meses
Melhor caso (MVP arquitetural)	~6–8 pessoa-meses	~5–7 pessoa-meses

Interpretação: As duas direções são **caras** porque envolvem **re-arquitetura do canal cliente ↔ servidor**, não um “rename”. Em geral, **Infotime → InfoLAB** tende a ser **ligeiramente mais cara** por introduzir **dois tiers** (Next + Nest) e **modelo de sessão** mais elaborado no edge.

7. Vantagens e desvantagens (por família de padrão) — recapitulação

Padrão tipo Infotime (SPA + API direta + Zod + Fastify)

- **Vantagens:** menos hops; CDN simples; Fastify eficiente em I/O; contrato Zod partilhável.
- **Desvantagens:** gestão cuidadosa de tokens/XSS; sem SSR nativo na mesma app.

Padrão tipo InfoLAB (Next + BFF + cookies + Express)

- **Vantagens:** cookies HttpOnly; proxy central; SSR/i18n/marketing no mesmo produto.
- **Desvantagens:** mais componentes em produção; latência extra; operação mais complexa.

8. Recomendações finais

1. **Não migrar stack inteira** só por homogeneidade visual — portar **padrões de UI** (CSS/componentes) custa menos que trocar Next↔Vite.
2. Qualquer migração **auth/BFF** deve incluir **revisão de segurança** e testes de regressão em **tenant isolation**.
3. Fixar **métricas de saída** (P95 API, erros 401/403 por tenant) antes e depois da migração.

9. Metadados do ficheiro

- Gerado para o repositório **infotime-web**.
- PDF derivado do Markdown homónimo na pasta `docs/`.

Fim do relatório.